

Assurance Assignment – Track 2: OWASP SAMM

Francesco Maria Fuligni | Role: SAMM specialist

Core practice – Security Testing (Verification): Dedicated security test cases for known-risk areas are run regularly. **Baseline: Level 2; after the improvement: Level 2.** All scores below use the assignment’s simplified rubric.

Actual baseline evidence. Before my core improvement, the team’s repository already contained dedicated authorization, path-traversal and login-throttling tests. GitHub Actions ran pytest on pull requests to `main` and pushes to `main` or feature branches. The link-access tests already checked a valid issued link (HTTP 200/PDF), four unissued-link patterns (HTTP 404/no PDF), and separation of two users’ documents. These were repeatable tests for known risks. The remaining Level 3 gap is systematic security coverage tracking.

Core improvement and observed evidence. Previously, the SHA-1 candidate tried in the CC work was only a manual check, and the negative-link tests did not assert absence of sensitive fixture values. I added a deterministic regression: `GET /api/get-version/640d87e741e6aa4c669a82a4cd304787960513ab` must return HTTP 404 without PDF data. I also made the four existing unissued-link cases assert that responses contain no fixture watermark secrets, intended recipients or filenames. Focused verification on 30 September 2026 passed all seven link-access cases and Ruff. The improvement is recorded in commit `fbae18a`; the evidence is the assertions and their observed outcomes.

Further Work option 1 – Second practice audited: Secure Build (Implementation). Builds are automated and reproducible; dependencies are pinned. **Baseline: Level 1; after improvements: scoped Level 2.** At the audited baseline, CI did not build the Docker image and Docker installed flexible Python dependency ranges instead of a tracked pinned set. A locally built image contained `cryptography` 50.0.2, while the unused local lockfile specified 50.0.1. This showed that the image did not follow that lockfile.

Further Work option 2 – Second real improvement. I addressed those build gaps with a CI Docker build/`import rmap` check and a tracked `requirements.txt` with 26 exactly versioned registry distributions. Docker and pytest install those requirements; CI regenerates the file and fails on drift. The Python base image is digest-pinned. Recorded build verification found all 26 versions matching the image; `pip check` and `import rmap` passed. Commits `8a1073f`, `f0cd7ae` and `f7e50c3` record these changes.

Limits and remaining build gap. The link tests use in-memory SQLite and the test-only `toy-eof` watermark; they establish selected regression outcomes, not general link unpredictability. The build score covers the server/Python runtime: system packages and the full Compose stack remain outside the reproducibility claim, and RMAP uses a release tag. The Docker check verifies an import, not full-stack startup. No dedicated dependency/image vulnerability gate is established; Secure Build Level 3 is not claimed.

Track 2 hours spent (self-reported): 9 hours (5 Core + 4 Further Work).